

מבנה מחשבים ספרתיים - תרגיל בית יבש 1

יוני 2026

מגיש 1:

שם: אביב לוי

ת"ז: 207033259

אימייל: aviv.levy@campus.technion.ac.il

מגיש 2:

שם: איל טיבן

ת"ז: 322585837

אימייל: eyal.tiben@campus.technion.ac.il

שאלה 1:

א.

$$CPI_{avg} = \sum_i (CPI_i \cdot Frequency_i) = 0.5 * 1 + 0.15 * 5 + 0.2 * 4 + 0.15 * 2 = 2.35$$

כלומר 2.35 מחזורי שעון בממוצע לפקודה.

ב. הגדרת ה CPI היא כמות מחזורי השעון לפקודה. אמנם במעבד החדש כל מחזור שעון ייקטן פי 2 אך ה CPI יישאר זהה.

ג. הנוסחה הכללית ל-Speedup:

$$SPEEDUP = \frac{t_{OLD}}{t_{NEW}} = \frac{CC_{OLD} \cdot CPI_{OLD} \cdot IC_{OLD}}{CC_{NEW} \cdot CPI_{NEW} \cdot IC_{NEW}}$$

נחשב את CPI_{NEW} :

$$CPI_{NEW} = 1 \cdot 0.5 \cdot 1.5 + \frac{0.15 \cdot 5}{2} + \frac{0.2 \cdot 4}{3} + 0.15 \cdot 2 = 1.69$$

בנוסף, $IC_{NEW} = IC_{OLD}$ ונתון $0.5 \cdot CC_{OLD} = CC_{NEW}$
נציב ונקבל:

$$SPEEDUP = \frac{2.35}{0.5 \cdot 1.69} = 2.778$$

שיפרנו פי 2.778 את זמן הריצה של התוכנית.

ד. $0.4 \cdot 0.5 = 0.2$ מהפקודות בכל התוכנית הן float במקום integer ולכן:

$$CPI_{new} = 1 \times 0.3 \times 1.5 + \frac{5 \times 0.15}{2} + \frac{4 \times 0.4}{3} + 2 \times 0.15 = 1.658$$

בנוסף, מתקיים:

$$CPI_{old} = 1.69, \quad CC_{old} = CC_{new}, \quad IC_{new} = 0.8 \cdot IC_{old}$$

כפי שחישבנו בסעיף הקודם ולכן:

$$\frac{CPU \text{ TIME}_{old}}{CPU \text{ TIME}_{new}} = \frac{IC_{old} \cdot CPI_{old} \cdot CC_{old}}{IC_{new} \cdot CPI_{new} \cdot CC_{new}} = \frac{1.69}{0.8 \cdot 1.658} = 1.27$$

כלומר קיבלנו כי הזמן החדש קטן יותר מהישן ולכן המעבד החדש עדיף.

שאלה 2:

א. נשים לב כי במעבד הפגום יכולה להתרחש בעיה כאשר קיימת פקודה בשלב ה-WB ובמקביל קיימת פקודה בשלב ה-ID כך ששתי הפקודות נדרשות לקרוא ולכתוב במקביל מקובץ הרגיסטרים.

נחלק למקרים:

R-type:

כל הפקודות מסוג זה שומרות את התוצאה ברגיסטר בשלב ה-WB ובנוסף קוראת משני הרגיסטרים בשלב ה-ID ולכן עלולים להתרחש התנגשויות.

I-type:

פקודות כמו lw מעדכנות את הרגיסטר בשלב ה-WB ופקודות כמו sw קוראות מרגיסטר בשלב ה-ID ולכן עלולים להתרחש התנגשויות.

J-type:

פקודות כמו jal שומרות את כתובת החזרה ברגיסטר בשלב ה-WB אבל כל פקודות הקפיצה כמו jal, jalr צריכות לקרוא משתנים מקובץ הרגיסטרים ולכן אם הן נמצאות בשלב ה-ID לא יתרחשו התנגשויות.

ב. ה-HDU יתוכנת באופן הבא:

אם מתקיימים שני התנאים במקביל:

1. הפקודה ב-WB מבצעת כתיבה האות Regwrite=1 מחוץ MEM/WB.
2. הפקודה ב-ID דורשת קריאה בתחילת שלב ה-ID לפני הכניסה לקובץ הרגיסטרים, בזיהוי סוג הפקודה, אם הפקודה היא מטיפוס R או I ולא פקודה מטיפוס J שלא זקוקה לקובץ הרגיסטרים.

הפעולה של ה-HDU:

כשהתנאי הלוגי מתקיים, ה-HDU ישהה את שלב ה-ID במחזור שעון 1 באמצעות bubble ויקפיא את הפקודה שאחריה כך שהיא תשאר בשלב ה-ID.

ג. בשביל למקסם את ה-CPI נשתמש בתכנית שבה קיימים רק פקודות מסוג R-type שלפי הסעיף הקודם עבור 2 פקודות R בשלב ה-WB ובשלב ה-ID תוצר התנגשות וכתוצאה מכך ה-HDU יבצע השהייה למשך מחזור שעון אחד.

CLOCK	IF	ID	EXE	MEM	WB
1	ADD				
2	ADD	ADD			
3	ADD	ADD	ADD		
4	ADD	ADD	ADD	ADD	
5	ADD	ADD	ADD	ADD	ADD
6	ADD	ADD	nop	ADD	ADD
7	ADD	ADD	nop	nop	ADD
8	ADD	ADD	nop	nop	nop
9	ADD	ADD	ADD	nop	nop
10	ADD	ADD	ADD	ADD	nop
11	ADD	ADD	ADD	ADD	ADD
12	ADD	ADD	nop	ADD	ADD
13	ADD	ADD	nop	nop	ADD
14	ADD	ADD	nop	nop	nop
...					

מתוך הטבלה, ניתן לזהות דפוס קבוע שחוזר על עצמו לאורך התוכנית: קבוצה של שלוש פקודות מצליחה להיכנס ל-Pipeline ולרוץ ברצף. לאחר מכן, ה-Pipeline נאלץ לעצור ולהכניס 3 בועות למשך 3 מחזורי שעות, משום ששלוש הפקודות הללו מגיעות בזו אחר זו לשלב ה-WB וחוסמות את הקריאה של הפקודות הבאות בתור. מיד כששלוש הפקודות מסיימות את שלב ה-WB, ה-Pipeline משתחרר ומאפשר לעוד 3 פקודות להיכנס, וחוזר חלילה. מכאן נובע שבכל בלוק זמן של 6 פעימות שעות, המעבד מסיים בפועל רק 3 פקודות. הנוסחה לחישוב ה-CPI הממוצע במצב היציב היא:

$$CPI = \frac{\text{TOTAL CYCLES}}{\text{COMPLETED INSTRUCTIONS}} = 2$$

מכיוון שקיימים רק structural hazards ללא data hazards, control hazards ומכיוון שבסעיף הקודם הראנו כי ה-HDU מבצע השהייה במשך מחזור שעות אחד, התכנית לעיל ממקסמת את ה-CPI.

שאלה 3:

א. גודל בלוק הוא 64 בתים ולכן ייצוג של offset הבלוק הוא: $\log_2(64) = 6$ ביטים.

$$\text{TOTAL CACHE BLOCKS} = \frac{\text{TOTAL CACHE SIZE}}{\text{BLOCK SIZE}} = \frac{2^{13}}{2^6} = 128 \text{ BLOCKS}$$

קיימים 4 ways לכן בכל set יש $\frac{128}{4} = 32$ בלוקים. כדי לאנדקס 32 סטים שונים, נדרשים $\log_2(32)$ ביטים. לכן, נדרשים 5 ביטים לשדה ה-set.

מרחב הכתובות הנתון הוא בן 64 ביט. ה-tag הוא מספר הביטים שנותרו לאחר הקצאת הביטים ל-set ול- $offset$: $64 - 5 - 6 = 53$. TAG = 53, לכן, נדרשים 53 ביטים לשדה ה-tag.

סך הכל נקבל:

TAG	SET	OFFSET
11-63	6-10	0-5

ב. עבור $x = 1$:

גודל בלוק הוא 64 בתים, כאשר גודל int הוא 4 בתים, לכן סך הכל מאוחסנים 16 אברי מערך בבלוק. לכן, בגישה לאיבר הראשון במערך A יתרחש miss וב-15 האיברים הבאים יתבצע hit. באותו אופן ובגלל שבמערך B אנחנו ניגשים רק לאינדקסים זוגיים, יתבצע miss לאחריו 8 hits.

$$\text{miss rate} = \frac{(1/16)N + (1/8)\frac{N}{2}}{N + \frac{N}{2}} = \frac{(2/16)}{1.5} = \frac{1}{12} = 8.33\%$$

עבור $x = 16$:

הפעם יהיו $N/16$ גישות זיכרון ל-A ובגלל שבלוק מכסה 16 איברים במערך כל הגישות יקבלו miss. בגלל שהקפיצות הן כפולות של 16 תמיד האינדקסים יהיו זוגיים וכל הגישות ל-A יתרחשו גם ב-B, כלומר גם ב-B כל הגישות יקבלו miss. כלומר:

$$\text{miss rate} = 100\%$$

עבור $x = 32, 2^{12}$:

מאותה סיבה עבור $x = 16$, מכיוון שהם כפולות של 16 נקבל כי:

$$\text{miss rate} = 100\%$$

ג. עבור fully associative לא יהיה קיים set. גודל בלוק הוא 64 בתים ולכן ייצוג של offset הבלוק הוא: $\log_2(64) = 6$ ביטים. לכן:

TAG	SET	OFFSET
6-63	--	0-5

נשים לב כי עבור תוכנית זו, כל ההחטאות התרחשו בגלל compulsory ולא בגלל קונפליקטים ולכן שיטת ה-fully associative לא תביא תוצאות טובות יותר, וה-miss rate יישאר זהה לכל ערך של x.